

■ Machine Learning Concepts & Topics Explained

Case Study 139: Soil Classification Project

A Complete Student-Friendly Guide explaining HOW, WHY, and WHEN each ML Concept is used in this project.

■ Table of Contents

1. [Supervised Machine Learning & Classification](#)
 2. [Multi-Class Classification](#)
 3. [Train-Test Split & Stratification](#)
 4. [Feature Scaling & Standardization \(StandardScaler\)](#)
 5. [Categorical Encoding \(LabelEncoder\)](#)
 6. [Exploratory Data Analysis \(EDA\) & Feature Correlation](#)
 7. [Stratified 5-Fold Cross-Validation](#)
 8. [Evaluation Metrics: Accuracy, Precision, Recall, F1-Score](#)
 9. [Confusion Matrix & Error Diagnostics](#)
 10. [Feature Importance \(Gini & Boosting Loss Reduction\)](#)
 11. [Bias-Variance Tradeoff & Overfitting vs Underfitting](#)
 12. [Model Serialization & Pipeline Deployment \(joblib\)](#)
-

1. Supervised Machine Learning & Classification

■ What is it?

Supervised Machine Learning is a method where the computer learns from **labeled examples**. We give the computer both the inputs (soil measurements) and the correct answers (the soil type). Once trained, it can predict the soil type for completely new, unseen soil measurements.

■ WHY do we use it in this project?

- In our project, we have 9 input features (*X*) like Nitrogen, Potassium, pH, Moisture, etc.
- We have 1 target output (*y*), which is the Soil Type (e.g. *Black Soil*, *Red Soil*).
- Supervised learning finds the mathematical relationship between the soil measurements and the soil type.

■ HOW does it work?

1. We feed thousands of soil test records into the ML model.
2. The model adjusts its internal mathematical equations until its predictions match the true soil labels.
3. We test the model on separate soil records to verify that it learned the real patterns rather than memorizing the data.

■ WHEN should you use it?

Use Supervised Classification when:

- You have historical data with known correct answers (labels).

- Your target variable is **discrete/categorical** (e.g., *Soil Type*, *Spam/Not Spam*), not continuous numbers like predicting house prices (which is Regression).
-

2. Multi-Class Classification

■ What is it?

In simple Binary Classification, there are only two outcomes (e.g., *Yes/No*, *Pass/Fail*). In **Multi-Class Classification**, there are **three or more categories**.

■ WHY do we use it in this project?

Our soil classification project needs to distinguish between **8 distinct soil types**:

1. Alluvial Soil
2. Black Soil
3. Red Soil
4. Laterite Soil
5. Clayey Soil
6. Sandy Loam
7. Saline Soil
8. Peaty Soil

■■ HOW does it work?

- Models like Decision Trees and Random Forests handle multi-class problems naturally by creating branch splits for different categories.
- Linear models like Logistic Regression use the **Softmax function** to calculate probabilities for all 8 categories simultaneously, ensuring all 8 probabilities add up to 100%.

■■ WHEN should you use it?

Use it whenever your problem requires predicting one category out of many possible groups.

3. Train-Test Split & Stratification

■ What is it?

Train-Test Split is the practice of dividing your full dataset into two separate subsets:

1. **Training Set (80% = 2,240 samples)**: Used exclusively to teach the models.
2. **Testing Set (20% = 560 samples)**: Kept in a locked vault during training, used only to evaluate the final exam score of the model.

■ WHY do we use it in this project?

If you test a student using the exact same questions they practiced, they might get 100% simply by memorizing. Testing the model on brand new, unseen data ensures it has genuinely learned how to identify soil types.

■ HOW does Stratification (stratify=y) work?

- Without stratification, a random split might accidentally put 90% of Peaty Soil into the training set and almost none into the test set.
- **Stratified splitting** ensures that every class has the exact same proportion (12.5% per soil class) in both the training and test sets.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
```

■ WHEN should you use it?

Always use train-test split for any machine learning project. Always enable `_CODE_PH_0_` when dealing with multi-class or imbalanced datasets.

4. Feature Scaling & Standardization (StandardScaler)

■ What is it?

Feature Scaling transforms all numerical columns so that they share the same scale (mean $\mu = 0$, standard deviation $\sigma = 1$).

■ WHY do we use it in this project?

Our soil measurements have vastly different units and numerical ranges:

- **Potassium (K):** Ranges from `_CODE_PH_0_` to `_CODE_PH_1_` (huge numbers).
- **Electrical Conductivity (EC):** Ranges from `_CODE_PH_0_` to `_CODE_PH_1_` (tiny numbers).
- **pH:** Ranges from `_CODE_PH_0_` to `_CODE_PH_1_`.

Without scaling, algorithms that compute distances (like KNN) will let Potassium dominate the math simply because the number 250 is much larger than 0.8, completely ignoring the crucial EC value!

■ HOW does it work?

For each value x , it subtracts the average (μ) and divides by the spread (σ):

$$z = \frac{x - \mu}{\sigma}$$

In our project:

- **KNN accuracy jumped from 92.50% to 98.04% (+5.54% gain)** with scaling.
- **Logistic Regression** converged smoothly without solver warnings.
- **Tree models (Decision Tree, Random Forest)** were unaffected because tree splits only care about relative order, not magnitude.

■ WHEN should you use it?

- **Mandatory for:** Distance-based models (KNN, SVM, K-Means), Gradient-based models (Logistic Regression, Neural Networks).
- **Optional/Not needed for:** Tree-based models (Decision Tree, Random Forest, Gradient Boosting).

5. Categorical Encoding (LabelEncoder)

■ What is it?

Machine learning algorithms only understand numbers. They cannot directly perform linear algebra on words like `_CODE_PH_0_` or `_CODE_PH_1_`. `_CODE_PH_2_` converts text categories into numbers:

Text Label	Encoded Integer ID
Alluvial Soil	0
Black Soil	1
Clayey Soil	2
Laterite Soil	3
Peaty Soil	4
Red Soil	5
Saline Soil	6
Sandy Loam	7

■ WHY and HOW do we use it?

We fit the encoder on the target column `_CODE_PH_0_`, convert it before training, and use `_CODE_PH_1_` in our Streamlit web app to display the original friendly soil name back to the user.

6. Exploratory Data Analysis (EDA) & Feature Correlation

■ What is it?

EDA is the investigative phase where you visualize distributions, detect outliers, and measure relationships between features before building models.

■ WHY do we use it in this project?

- To verify that our dataset has zero missing values and zero duplicate rows.
- To understand how pH, nutrients, and moisture differ across soil types.
- To check Pearson correlation coefficients and ensure there is no severe multicollinearity.

■ HOW does it work in our code?

- **Boxplots:** Show the median, quartiles, and range of each nutrient for every soil category.
- **Correlation Heatmap:** Computes the Pearson correlation matrix `<i>r</i>` in `[-1, 1]` between all 9 features.
- **Acidity vs Salinity Scatter:** Plots pH against Electrical Conductivity, showing clear clustering of Saline and Peaty soils.

7. Stratified 5-Fold Cross-Validation

■ What is it?

Cross-validation is a technique to test model stability by splitting the dataset into 5 equal parts (folds).

```
Fold 1: [ Test ][ Train ][ Train ][ Train ][ Train ] -> Score 1
Fold 2: [ Train ][ Test ][ Train ][ Train ][ Train ] -> Score 2
Fold 3: [ Train ][ Train ][ Test ][ Train ][ Train ] -> Score 3
Fold 4: [ Train ][ Train ][ Train ][ Test ][ Train ] -> Score 4
Fold 5: [ Train ][ Train ][ Train ][ Train ][ Test ] -> Score 5
Average Score = Mean of all 5 scores ± Standard Deviation
```

■ WHY do we use it in this project?

A single train-test split might get lucky or unlucky depending on which samples were chosen. 5-Fold CV tests the model 5 separate times on different partitions and averages the result (e.g. **98.54% ± 0.86%** for Logistic Regression), proving the model is genuinely reliable across all data.

8. Evaluation Metrics: Accuracy, Precision, Recall, F1-Score

■ The 4 Core Metrics Explained Simply:

1. Accuracy

- **Formula:** $\frac{\text{Correct Predictions}}{\text{Total Predictions}}$
- **What it means:** Overall percentage of soil samples classified correctly (e.g. 99.29%).

2. Precision (Quality of positive predictions)

- **Formula:** $\frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$
- **What it means:** When the model predicts "Black Soil", how often is it actually Black Soil? High precision means very few false alarms.

3. Recall / Sensitivity (Quantity of actual samples caught)

- **Formula:** $\frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$
- **What it means:** Out of all the real Black Soil samples in the field, what percentage did the model correctly identify? High recall means very few missed soils.

4. F1-Score (The Balanced Harmonic Mean)

- **Formula:** $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$
- **What it means:** The single most reliable metric because it penalizes models that have high precision but poor recall (or vice versa).

Macro F1 vs. Weighted F1:

- **Macro F1:** Computes the unweighted average F1 across all 8 classes equally.
- **Weighted F1:** Weights each class score by the number of samples in that class.

9. Confusion Matrix & Error Diagnostics

■ What is it?

An 8×8 grid showing exactly where the model was right (diagonal cells) and where it got confused (off-diagonal cells).

■ Key Finding from our Project (Case Study Q4):

- **100% Perfect Classes:** Saline Soil, Peaty Soil, Black Soil, Laterite Soil, Clayey Soil had **0 errors**.
- **Most Confused Pair: Sandy Loam vs. Red Soil** (3 samples of Sandy Loam were misclassified as Red Soil).
- **Why?** Both soils share low moisture ($12\text{--}22\%$), low organic carbon ($<1.2\%$), and mild acidity, making their geochemical boundary slightly closer than other soils.

10. Feature Importance (Gini & Boosting Loss Reduction)

■ What is it?

Feature Importance ranks which soil properties helped the ML models make the most accurate decisions.

■ Key Finding from our Project (Case Study Q2):

1. **Electrical Conductivity (18.84%):** Sharpest discriminator for Saline soils.
2. **Organic Matter (17.56%):** Sharpest discriminator for Peaty bog soils.
3. **Moisture (16.10%):** Separates heavy waterlogged clays from dry sandy loams.
4. **pH (10.14%):** Separates acidic laterites from alkaline black/saline soils.
5. **Macronutrients (P, K, N):** Fine-tune fertility identification.
6. **Temperature (0.31%):** Least influential overall.

11. Bias-Variance Tradeoff & Overfitting vs Underfitting

■ What are they?

- **Underfitting (High Bias):** Model is too simple; cannot capture patterns in training data (like drawing a straight line through a circle).
- **Overfitting (High Variance):** Model is too complex; it memorizes noise in the training set and fails on new test data.
- **Balanced Sweet Spot:** Models like Random Forest and Regularized Logistic Regression that generalize well to new data.

■ How we prevented overfitting in this project:

1. Limited tree depth (**max_depth=10** in Decision Tree, **_CODE_PH_1_** in Random Forest).
2. Used 120 randomized trees in Random Forest to average out individual tree variance.
3. Applied L2 weight penalty in Logistic Regression.
4. Validated on 5-Fold Stratified Cross-Validation.

12. Model Serialization & Pipeline Deployment (joblib)

■ What is it?

After training a machine learning model, keeping it in memory is not enough. Serialization writes the trained mathematical weights and rules to binary `_CODE_PH_0_` files on disk.

■ HOW is it used in our Streamlit Web App?

1. `train_and_evaluate.py` saves `_CODE_PH_1_`, `_CODE_PH_2_`, `_CODE_PH_3_`, and `_CODE_PH_4_` into the `_CODE_PH_5_` directory.
2. When a user visits `_CODE_PH_0_`, `_CODE_PH_1_` loads these files via `_CODE_PH_2_` in under 0.1 seconds.
3. When the user moves the sliders, the pre-trained model computes instant predictions without needing to retrain from scratch.